

Atividade 1 — Listas

```
# Criamos uma lista vazia para armazenar os números
numeros = []

# Repetimos 10 vezes para solicitar os números
for i in range(10):

    # Solicita um número inteiro ao usuário
    numero = int(input(f"Digite o {i + 1}º número: "))

    # Adiciona o número na lista
    numeros.append(numero)

# Exibe todos os números digitados
print("\nNúmeros digitados:")
print(numeros)

# Calcula a soma de todos os números
soma = sum(numeros)

# Calcula a média
media = soma / len(numeros)

# Encontra o maior número
maior = max(numeros)

# Encontra o menor número
menor = min(numeros)

# Conta quantos números são pares
quantidade_pares = 0

for numero in numeros:

    # Um número é par quando o resto da divisão por 2 é 0
    if numero % 2 == 0:
```

```
        quantidade_pares += 1

# Exibe os resultados
print("\n===== RESULTADOS =====")
print(f"Soma: {soma}")
print(f"Média: {media:.2f}")
print(f"Maior número: {maior}")
print(f"Menor número: {menor}")
print(f"Quantidade de números pares: {quantidade_pares}")
```

Atividade 2 — Matrizes

```
# Criamos uma matriz vazia
matriz = []

# Vamos criar 3 linhas
for i in range(3):

    # Criamos uma lista para representar a linha atual
    linha = []

    # Cada linha terá 3 colunas
    for j in range(3):

        # Solicita o valor ao usuário
        valor = int(input(
            f"Digite o valor da posição [{i}][{j}]: "
        ))

        # Adiciona o valor na linha
        linha.append(valor)

    # Adiciona a linha completa na matriz
    matriz.append(linha)

# EXIBIR A MATRIZ

print("\n===== MATRIZ =====")

for linha in matriz:

    # Exibe cada linha da matriz
```

```
    # Exibe cada linha da matriz
    print(linha)

# SOMA DOS ELEMENTOS

soma = 0

# Percorre cada linha
for linha in matriz:

    # Percorre cada elemento da linha
    for valor in linha:

        # Adiciona o valor à soma
        soma += valor

# DIAGONAL PRINCIPAL

soma_diagonal = 0

# Na diagonal principal:
# linha e coluna possuem o mesmo índice
#
# [0][0]
# [1][1]
# [2][2]

for i in range(3):

    soma_diagonal += matriz[i][i]

# MAIOR ELEMENTO

# Começamos considerando o primeiro elemento
maior = matriz[0][0]

# Percorremos toda a matriz
for linha in matriz:

    for valor in linha:

        # Se encontrarmos um valor maior,
        # atualizamos a variável maior
        if valor > maior:
```

```
        if valor > maior:
            maior = valor

# RESULTADOS

print("\n===== RESULTADOS =====")
print(f"Soma dos elementos: {soma}")
print(f"Soma da diagonal principal: {soma_diagonal}")
print(f"Maior elemento: {maior}")
```

Atividade 3 — Dicionário

```
# Criamos um dicionário vazio
livro = {}

# CADASTRO DO LIVRO

# Solicita o título
livro["titulo"] = input("Digite o título do livro: ")

# Solicita o autor
livro["autor"] = input("Digite o autor do livro: ")

# Solicita o ano
livro["ano"] = int(input("Digite o ano do livro: "))

# Solicita o preço
livro["preco"] = float(input("Digite o preço do livro: "))

# EXIBIR OS DADOS

print("\n===== DADOS DO LIVRO =====")

print(f"Título: {livro['titulo']}")
print(f"Autor: {livro['autor']}")
print(f"Ano: {livro['ano']}")
print(f"Preço: R$ {livro['preco']:.2f}")

# ALTERAR O PREÇO

novo_preco = float(input("\nDigite o novo preço: "))
```

```
# Alteramos o valor associado à chave "preco"
livro["preco"] = novo_preco

# ADICIONAR CATEGORIA

# Criamos uma nova chave no dicionário
livro["categoria"] = input("Digite a categoria do livro: ")

# EXIBIR DADOS ATUALIZADOS

print("\n==== LIVRO ATUALIZADO =====")

# Percorremos todas as chaves e valores
for chave, valor in livro.items():

    print(f"{chave}: {valor}")
```

Atividade 4 — Dataclass

```
# Importamos o dataclass
from dataclasses import dataclass

# Criamos a dataclass Aluno
@dataclass
class Aluno:

    # Atributos do aluno
    nome: str
    matricula: int
    nota1: float
    nota2: float

    # Método responsável por calcular a média
    def media(self):

        # Soma as duas notas e divide por 2
        return (self.nota1 + self.nota2) / 2

# CRIANDO UM ALUNO
```

```
aluno = Aluno(
    "Maria",
    12345,
    8.0,
    9.0
)

# EXIBINDO OS DADOS

print("==== ALUNO ====")

print(f"Nome: {aluno.nome}")
print(f"Matrícula: {aluno.matricula}")
print(f"Nota 1: {aluno.nota1}")
print(f"Nota 2: {aluno.nota2}")

# Chamamos o método media()
print(f"Média: {aluno.media():.2f}")
```

Atividade 5 — Lista de objetos

```
# Importamos o dataclass
from dataclasses import dataclass

# Criamos a classe Aluno
@dataclass
class Aluno:

    nome: str
    matricula: int
    nota1: float
    nota2: float

    # Método para calcular a média
    def media(self):

        return (self.nota1 + self.nota2) / 2

# Criamos uma lista vazia para armazenar os alunos
alunos = []
```

```
# CADASTRAR 5 ALUNOS

for i in range(5):

    print(f"\n===== ALUNO {i + 1} =====")

    # Solicita os dados
    nome = input("Nome: ")
    matricula = int(input("Matrícula: "))
    nota1 = float(input("Nota 1: "))
    nota2 = float(input("Nota 2: "))

    # Cria um objeto da classe Aluno
    aluno = Aluno(
        nome,
        matricula,
        nota1,
        nota2
    )

    # Adiciona o objeto à lista
    alunos.append(aluno)

# EXIBIR MÉDIA DOS ALUNOS

print("\n===== MÉDIA DOS ALUNOS =====")

for aluno in alunos:

    print(
        f"{aluno.nome} - "
        f"Média: {aluno.media():.2f}"
    )

# ALUNOS APROVADOS

print("\n===== ALUNOS APROVADOS =====")

for aluno in alunos:

    # A aprovação ocorre quando a média é >= 7
    if aluno.media() >= 7.0:
```

```
        print(
            f"{aluno.nome} - "
            f"Média: {aluno.media():.2f}"
        )

# ALUNO COM MAIOR MÉDIA

# Começamos considerando o primeiro aluno
maior_aluno = alunos[0]

# Percorremos os demais alunos
for aluno in alunos:

    # Verificamos se a média dele é maior
    if aluno.media() > maior_aluno.media():

        maior_aluno = aluno

# Exibe o resultado
print("\n===== MAIOR MÉDIA =====")

print(f"Aluno: {maior_aluno.nome}")
print(f"Média: {maior_aluno.media():.2f}")
```

Desafio Integrador — Sistema de Estoque

```
# Importamos o dataclass
from dataclasses import dataclass

# CLASSE PRODUTO

@dataclass
class Produto:

    # Informações do produto
    codigo: int
    nome: str
    preco: float
    quantidade: int

    # Método para calcular o valor
    # total desse produto no estoque
```

```
# total desse produto no estoque
def valor_estoque(self):

    return self.preco * self.quantidade

# LISTA DE PRODUTOS

# Essa lista armazenará todos os produtos cadastrados
produtos = []

# MENU PRINCIPAL

while True:

    print("\n===== SISTEMA DE ESTOQUE =====")
    print("1 - Cadastrar produto")
    print("2 - Listar produtos")
    print("3 - Buscar produto")
    print("4 - Valor total do estoque")
    print("5 - Produto mais caro")
    print("0 - Sair")

    # Solicita uma opção ao usuário
    opcao = input("Escolha: ")

    # OPÇÃO 1 - CADASTRAR

    if opcao == "1":

        print("\n===== CADASTRAR PRODUTO =====")

        # Solicita os dados
        codigo = int(input("Código: "))
        nome = input("Nome: ")
        preco = float(input("Preço: "))
        quantidade = int(input("Quantidade: "))

        # Cria um objeto Produto
        produto = Produto(
            codigo,
            nome,
            preco,
```

```
        quantidade
    )

    # Adiciona o produto à lista
    produtos.append(produto)

    print("Produto cadastrado com sucesso!")

# OPÇÃO 2 - LISTAR

elif opcao == "2":

    print("\n===== PRODUTOS =====")

    # Verifica se existem produtos
    if len(produtos) == 0:

        print("Nenhum produto cadastrado.")

    else:

        # Percorre todos os produtos
        for produto in produtos:

            print("\n-----")

            print(f"Código: {produto.codigo}")
            print(f"Nome: {produto.nome}")
            print(f"Preço: R$ {produto.preco:.2f}")
            print(f"Quantidade: {produto.quantidade}")

            # Calcula o valor do estoque daquele produto
            print(
                f"Valor em estoque: "
                f"R$ {produto.valor_estoque():.2f}"
            )

# OPÇÃO 3 - BUSCAR PRODUTO

elif opcao == "3":

    codigo_busca = int(
        input("Digite o código do produto: ")
    )
```

```
# Variável para controlar se encontramos o produto
encontrado = False

# Percorre todos os produtos
for produto in produtos:

    # Verifica se o código é igual ao procurado
    if produto.codigo == codigo_busca:

        print("\n===== PRODUTO ENCONTRADO =====")

        print(f"Código: {produto.codigo}")
        print(f"Nome: {produto.nome}")
        print(f"Preço: R$ {produto.preco:.2f}")
        print(f"Quantidade: {produto.quantidade}")

        encontrado = True

        # Não precisamos continuar procurando
        break

# Se nenhum produto foi encontrado
if not encontrado:

    print("Produto não encontrado.")

# OPÇÃO 4 - VALOR TOTAL

elif opcao == "4":

    # Começamos o total em zero
    valor_total = 0

    # Percorremos todos os produtos
    for produto in produtos:

        # Somamos o valor de cada produto
        valor_total += produto.valor_estoque()

    print("\n===== VALOR TOTAL =====")

    print(
```

```
        f"Valor total do estoque: "  
        f"R$ {valor_total:.2f}"  
    )  
  
# OPÇÃO 5 - PRODUTO MAIS CARO  
  
elif opcao == "5":  
  
    # Verificamos se existe algum produto  
    if len(produtos) == 0:  
  
        print("Nenhum produto cadastrado.")  
  
    else:  
  
        # Começamos considerando o primeiro produto  
        produto_mais_caro = produtos[0]  
  
        # Percorremos todos os produtos  
        for produto in produtos:  
  
            # Comparamos os preços  
            if produto.preco > produto_mais_caro.preco:  
  
                produto_mais_caro = produto  
  
        print("\n===== PRODUTO MAIS CARO =====")  
  
        print(f"Nome: {produto_mais_caro.nome}")  
        print(  
            f"Preço: "  
            f"R$ {produto_mais_caro.preco:.2f}"  
        )  
  
# OPÇÃO 0 - SAIR  
  
elif opcao == "0":  
  
    print("Programa encerrado.")  
  
    # Encerra o while  
    break
```

```
# OPÇÃO INVÁLIDA

else:

    print("Opção inválida!")
```